

Experiment 06:

Laboratory Exercise

❖ Dataset 1: Iris

1. Input and Target Variable

Code:

```
X=df[['SepalLengthCm']]
y=df['PetalLengthCm']

print('Input : SepalLengthCm')
print('Target: PetalLengthCm')
```

Output:

```
*** Input : SepalLengthCm
    Target: PetalLengthCm
```

2. Training and Testing Data

Code:

```
from sklearn.model_selection import train_test_split

# X = features, y = target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

print('Training samples: ',len(X_train))
print('Testing samples : ',len(X_test))
```

Output:

```
*** Training samples: 120
    Testing samples : 30
```

3. Linear Regression Model

Code:

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
print(model)
```

Output:

```
... LinearRegression()
```

4. Model Training

Code:

```
model.fit(X_train,y_train)
print('Model training is completed.')
```

Output:

```
... Model training is completed.
```

5. Intercept and Slope

Code:

```
intercept=model.intercept_
slope=model.coef_[0]

print('Intercept (b0) :',intercept)
print('Slope (b1) :',slope)
print('Regression Equation: PetalLengthCm=',intercept,'+',slope,'* SepalLengthCm')
```

Output:

```
Intercept (b0) : -6.919754643134638
Slope (b1) : 1.832836834279381
Regression Equation: PetalLengthCm= -6.919754643134638 + 1.832836834279381 * SepalLengthCm
```

6. Generate Predictions

Code:

```
y_pred=model.predict(X_test)
print('predicted values:')
print(y_pred)
```

Output:

```
... predicted values:
[4.26055005 3.52741531 7.19308898 4.07726636 5.54353583 2.97756426
 3.34413163 5.72681951 4.44383373 3.710699 4.99368478 1.87786216
 3.16084795 2.06114584 2.42771321 4.62711741 4.99368478 3.34413163
 3.52741531 4.8104011 1.69457848 4.26055005 2.24442953 4.8104011
 7.55965635 5.36025215 5.36025215 5.54353583 1.87786216 1.87786216]
```

7. Actual and Predicted Values

Code:

```
result=pd.DataFrame({'Actual':y_test.values,'Predicted':y_pred})
result.head()
```

Output:

```
...
   Actual Predicted
0      4.7    4.260550
1      1.7    3.527415
2      6.9    7.193089
3      4.5    4.077266
4      4.8    5.543536
```

8. Mean Absolute Error

Code:

```
mae=mean_absolute_error(y_test,y_pred)
print('Mean Absolute Error:',mae)
```

Output:

```
... Mean Absolute Error: 0.6167659664774572
```

9. Mean Squared Error**Code:**

```
mse=mean_squared_error(y_test,y_pred)
print('Mean Square Error:',mse)
```

Output:

```
... Mean Square Error: 0.5965145177566363
```

10. R-Squared Score**Code:**

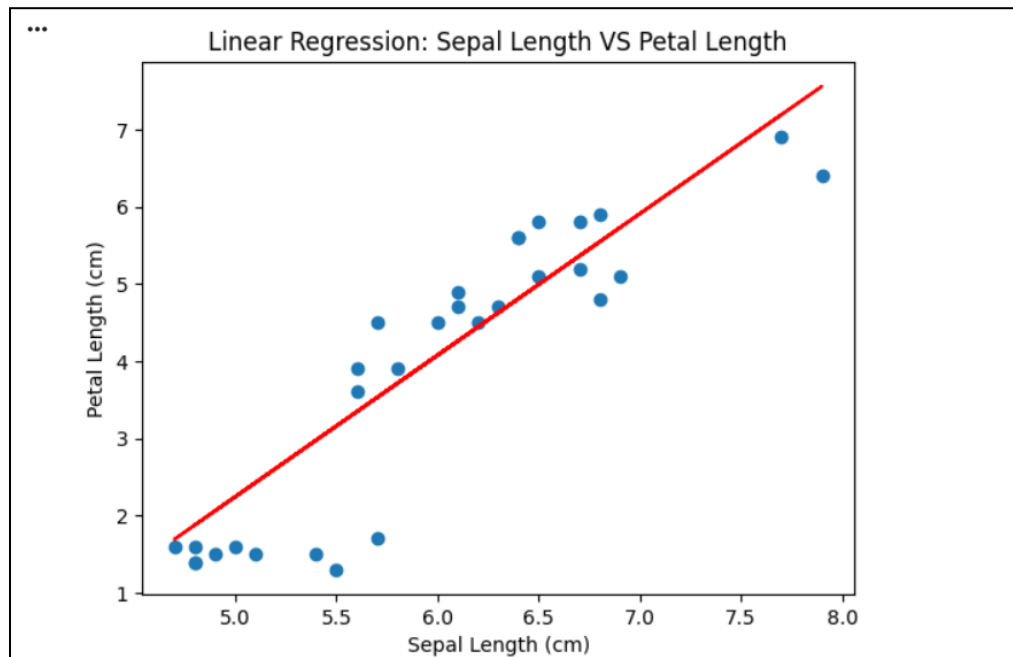
```
r2=r2_score(y_test,y_pred)
print('R-squared:',r2)
```

Output:

```
... R-squared: 0.8179909257095002
```

11. Regression Line**Code:**

```
plt.scatter(X_test,y_test)
plt.plot(X_test,y_pred,color='red',label='regression Line')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Petal Length (cm)')
plt.title('Linear Regression: Sepal Length VS Petal Length')
plt.show()
```

Output:**12. New Input Prediction****Code:**

```
new_value=pd.DataFrame({'SepalLengthCm':[5.5]})
prediction=model.predict(new_value)
print('Input Sepal Length: ',5.5)
print('Predicted Petal Length:',prediction[0])
```

Output:

```
... Input Sepal Length: 5.5
Predicted Petal Length: 3.160847945401958
```

13. Model Performance**Code:**

```
print('-----Model Performance')
print('MAE      :',mae)
print('MSE      :',mse)
print('R-squared :',r2)
```

Output:

```
... -----Model Performance
MAE      : 0.6167659664774572
MSE      : 0.5965145177566363
R-squared : 0.8179909257095002
```

❖ Dataset 2: California Housing**1. Input and Target Variable****Code:**

```
X=df[['MedInc']]
y=df['MedHouseVal']

print('Input : Median House Income')
print('Target: Median House Value')
```

Output:

```
... Input : Median House Income
Target: Median House Value
```

2. Training and Testing Data**Code:**

```
from sklearn.model_selection import train_test_split

# X = features, y = target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

print('Training samples: ',len(X_train))
print('Testing samples : ',len(X_test))
```

Output:

```
... Training samples: 16512
Testing samples : 4128
```

3. Linear Regression Model

Code:

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
print(model)
```

Output:

```
... LinearRegression()
```

4. Model Training

Code:

```
model.fit(X_train,y_train)
print('Model training is completed.')
```

Output:

```
... Model training is completed.
```

5. Intercept and Slope

Code:

```
intercept=model.intercept_
slope=model.coef_[0]

print('Intercept (b0) :',intercept)
print('Slope (b1) :',slope)
print('Regression Equation: MedHouseVal=',intercept,'+',slope,'* MedInc')
```

Output:

```
... Intercept (b0) : 0.4445972916907879
   Slope (b1) : 0.4193384939381271
   Regression Equation: MedHouseVal= 0.4445972916907879 + 0.4193384939381271 * MedInc
```

6. Generate Predictions

Code:

```
y_pred=model.predict(X_test)
print('predicted values:')
print(y_pred)
```

Output:

```
... predicted values:
[1.14958917 1.50606882 1.90393718 ... 4.31500772 1.612455 1.93412956]
```

7. Actual and Predicted Values

Code:

```
result=pd.DataFrame({'Actual':y_test.values,'Predicted':y_pred})
result.head()
```

Output:

```
...
   Actual Predicted
0  0.47700    1.149589
1  0.45800    1.506069
2  5.00001    1.903937
3  2.18600    2.850594
4  2.78000    2.006633
```

8. Mean Absolute Error

Code:

```
mae=mean_absolute_error(y_test,y_pred)
print('Mean Absolute Error:',mae)
```

Output:

```
... Mean Absolute Error: 0.629908653009376
```

9. Mean Squared Error

Code:

```
mse=mean_squared_error(y_test,y_pred)
print('Mean Square Error:',mse)
```

Output:

```
... Mean Square Error: 0.7091157771765549
```

10. R-Squared Score

Code:

```
r2=r2_score(y_test,y_pred)
print('R-squared:',r2)
```

Output:

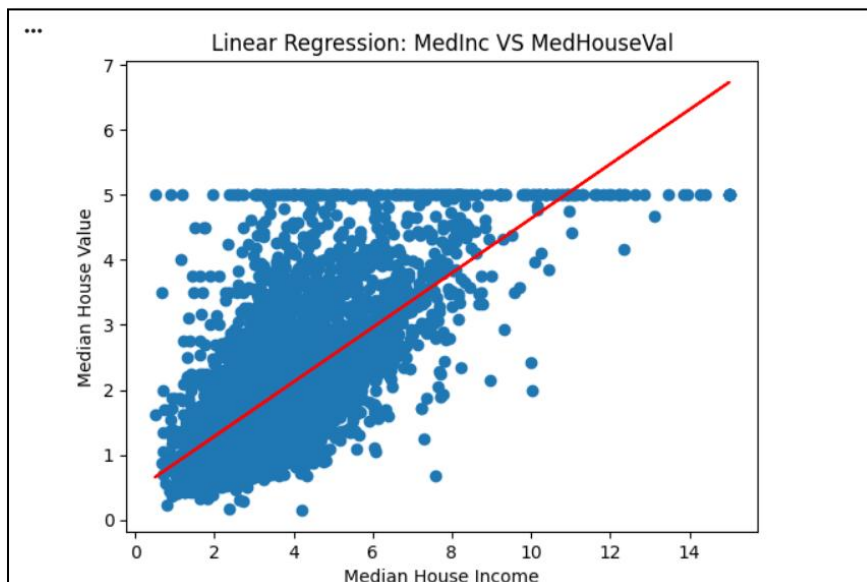
```
... R-squared: 0.45885918903846656
```

11. Regression Line

Code:

```
plt.scatter(X_test,y_test)
plt.plot(X_test,y_pred,color='red',label='regression Line')
plt.xlabel('Median House Income')
plt.ylabel('Median House Value')
plt.title('Linear Regression: MedInc VS MedHouseVal')
plt.show()
```

Output:



12. Model Performance

Code:

```
print('-----Model Performance')
print('MAE      :',mae)
print('MSE      :',mse)
print('R-squared :',r2)
```

Output:

```
... -----Model Performance
MAE      : 0.629908653009376
MSE      : 0.7091157771765549
R-squared : 0.45885918903846656
```

B. Questions:

1. Download iris.csv dataset containing numerical attributes. Select one input and one target attribute, implement Linear Regression and write your observations.

1. Input and Target Variable

Code:

```
X=df[['PetalLengthCm']]
y=df['PetalWidthCm']

print('Input : SepalLengthCm')
print('Target: PetalLengthCm')
```

Output:

```
... Input : PetalLengthCm
Target: PetalWidthCm
```

2. Training and Testing Data

Code:

```
from sklearn.model_selection import train_test_split

# X = features, y = target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
print('Training samples: ',len(X_train))
print('Testing samples : ',len(X_test))
```

Output:

```
... Training samples: 120
    Testing samples : 30
```

3. Linear Regression Model**Code:**

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
print(model)
```

Output:

```
... LinearRegression()
```

4. Model Training**Code:**

```
model.fit(X_train,y_train)
print('Model training is completed.')
```

Output:

```
... Model training is completed.
```

5. Intercept and Slope**Code:**

```
intercept=model.intercept_
slope=model.coef_[0]

print('Intercept (b0) :',intercept)
print('Slope (b1) :',slope)
print('Regression Equation: PetalWidthCm=',intercept,'+',slope,'*
PetalLengthCm')
```

Output:

```
... Intercept (b0) : -0.3609364607144494
Slope (b1) : 0.4140674609562574
Regression Equation: PetalWidthCm= -0.3609364607144494 + 0.4140674609562574 * PetalLengthCm
```

6. Generate Predictions**Code:**

```
y_pred=model.predict(X_test)
print('predicted values:')
print(y_pred)
```

Output:

```
... predicted values:
[1.58518061 0.34297822 2.49612902 1.50236711 1.62658735 0.26016473
 1.1297064 1.75080759 1.50236711 1.25392664 1.75080759 0.21875798
 0.17735124 0.26016473 0.26016473 1.58518061 2.04065481 1.25392664
 1.50236711 1.95784132 0.30157148 1.6679941 0.30157148 1.95784132
 2.28909529 1.79221434 2.04065481 2.08206156 0.21875798 0.30157148]
```

7. Actual and Predicted Values**Code:**

```
result=pd.DataFrame({'Actual':y_test.values,'Predicted':y_pred})
result.head()
```

Output:

```
... Actual Predicted
```

0	1.2	1.585181
1	0.3	0.342978
2	2.3	2.496129
3	1.5	1.502367
4	1.4	1.626587

8. Mean Absolute Error

Code:

```
mae=mean_absolute_error(y_test,y_pred)
print('Mean Absolute Error:',mae)
```

Output:

```
... Mean Absolute Error: 0.16815085976493724
```

9. Mean Squared Error

Code:

```
mse=mean_squared_error(y_test,y_pred)
print('Mean Square Error:',mse)
```

Output:

```
... Mean Square Error: 0.04557819994873102
```

10. R-Squared Score

Code:

```
r2=r2_score(y_test,y_pred)
print('R-squared:',r2)
```

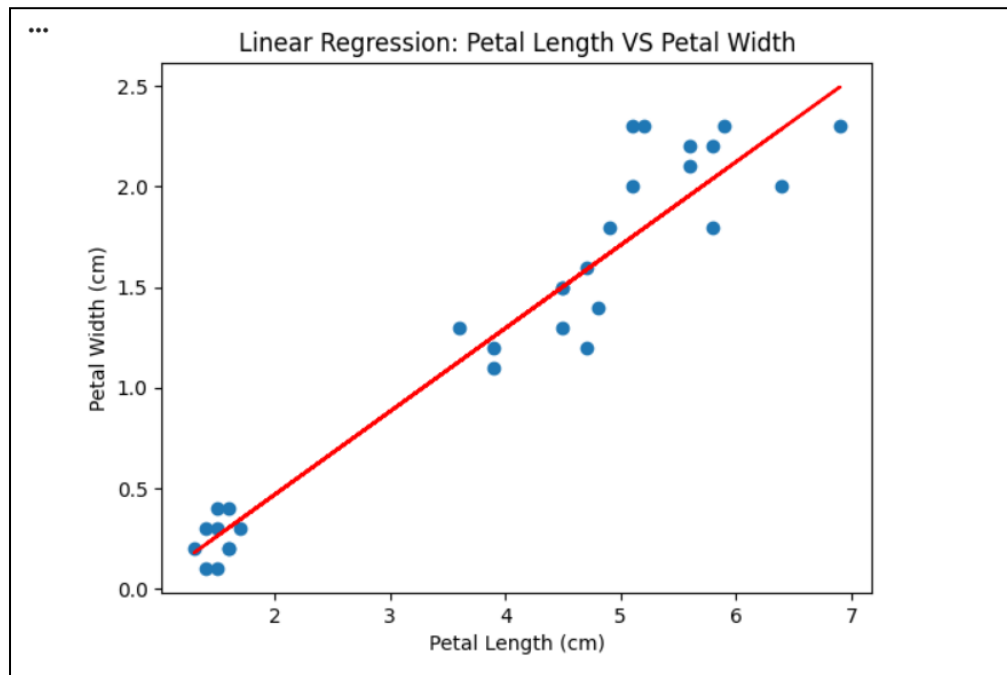
Output:

```
... R-squared: 0.9282973309202085
```

11. Regression Line

Code:

```
plt.scatter(X_test,y_test)
plt.plot(X_test,y_pred,color='red',label='regression Line')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('Linear Regression: Petal Length VS Petal Width')
plt.show()
```

Output:**12. Model Performance****Code:**

```
print('-----Model Performance')
print('MAE      :',mae)
print('MSE      :',mse)
print('R-squared :',r2)
```

Output:

```
*** -----Model Performance
MAE      : 0.16815085976493724
MSE      : 0.04557819994873102
R-squared : 0.9282973309202085
```

Observation:

1. **Petal Length** was selected as the input variable and **Petal Width** was selected as the target variable.
2. The dataset was divided into **120 training samples and 30 testing samples** using an 80:20 split.
3. The obtained regression equation is **PetalWidthCm = -0.3609 + 0.4141 × PetalLengthCm**, showing a positive relationship between petal length and petal width.

4. The predicted values were reasonably close to the actual values for several test samples. For example, an actual value of **1.2** was predicted as **1.5852**, while an actual value of **2.3** was predicted as **2.4961**.
 5. The **Mean Absolute Error (MAE)** was approximately **0.1682**, showing the average prediction error of the model.
 6. The **Mean Squared Error (MSE)** was approximately **0.0456**, indicating that the prediction errors were relatively small.
 7. The **R-squared value** was approximately **0.9283**, which shows that the model explains a large part of the variation in Petal Width.
 8. The regression graph shows that the data points generally follow an **upward trend**, confirming the positive relationship between Petal Length and Petal Width.
 9. For a new input **Petal Length = 5.5 cm**, the model predicted the **Petal Width = 1.9163 cm**.
-

2. Predict a numerical value for a new input using the trained Linear Regression model and interpret the result.

Output:

```
... Input Petal Length: 5.5  
Predicted Petal Width: 1.9164345745449665
```